

Cracking Frontend Interviews in 2026 — Phase 2 (React + LLD) 🧵

If JavaScript is the foundation,

React is where interviews get serious.

🔥 PHASE 2 — React (Beginner → Advanced)

- ▶ Why React? JSX & Babel
- ▶ Functional vs Class Components
- ▶ Props vs State
- ▶ React Lifecycle (important for interviews)
- ▶ Hooks — useState, useEffect, useRef, useContext, useReducer
- ▶ Custom Hooks (highly asked)
- ▶ useEffect vs useLayoutEffect
- ▶ React.memo vs useMemo vs useCallback
- ▶ Virtual DOM → Reconciliation → Fiber
- ▶ Why keys matter in lists
- ▶ Controlled vs Uncontrolled components
- ▶ Error Boundaries

- ▶ Portals & Refs

Advanced React

- ▶ State Management — Context API vs Redux vs Zustand

- ▶ How Redux works internally

- ▶ API Handling — fetch vs axios

- ▶ React Query (TanStack Query)

- ▶ Rendering — CSR vs SSR vs SSG vs ISR

- ▶ Hydration & Hydration issues

- ▶ TypeScript with React

- ▶ Micro Frontend Architecture

- ▶ Design Patterns in React

LLD in React (GAME CHANGER) - Don't just read — build these.

- ▶ Counter (with edge cases)

- ▶ Stopwatch / Timer

- ▶ Todo App (with clean state logic)

- ▶ Search (Debounce + Throttle + Infinite Scroll)

► Accordion / Modal / Tabs / Autocomplete

► Form with validation (no library)

► Virtualized List (10k+ rows efficiently)

► Drag & Drop UI

★ Most Asked:

► File Explorer (Nested Tree structure)

→ recursion + state + performance

 Practice Resources:

► greatfrontend.com

► bigfrontend.dev

► Learn patterns from → patterns.dev

If you can build these from scratch,

you're already ahead of 80% candidates.

Next: Phase 3 — SEO & Performance ⚡

 Repost to help someone preparing for frontend interviews.

[#ReactJS](#) [#FrontendDevelopment](#) [#WebDevelopment](#) [#InterviewPrep](#) [#LLD](#) [#JavaScript](#)
[#SoftwareEngineering](#) [#CareerGrowth](#)

✗ Got rejected in a Frontend interview — but learned something important.

Frontend interviews aren't really about React... they're about how strong your JavaScript is.

Recently went through a Frontend Developer interview process and here's a round-wise breakdown with some of the most asked questions 📌

- ♦ JavaScript (Most Important Round)

This is where most candidates struggle.

1. What is closure? Where have you used it?
2. Explain event loop with execution order
3. Implement debounce/throttle in JavaScript
4. How does "this" behave in different contexts?
5. Promise chaining vs async/await

- ♦ Round 2: React Deep Dive

1. Why do components re-render?
2. useMemo vs useCallback vs React.memo
3. How does useEffect lifecycle work?

4. How do you prevent unnecessary renders?

5. Real-world state management approach

♦ Round 3: Machine Coding

1. Build a debounced search / autocomplete

2. Handle API calls with proper states

3. Focus on clean architecture & reusability

4. Edge cases + performance considerations

♦ Round 4: Frontend System Design

1. Design a scalable UI (dashboard/feed)

2. Folder structure & code organization

3. API handling and caching

4. Performance optimization techniques

♦ Round 5: Hiring Manager Round

1. Deep dive into your project

2. Why did you choose certain approaches

3. Challenges and trade-offs

4. Ownership and decision making

💡 Biggest takeaway:

Frameworks change, but strong fundamentals stay.

Don't forget to like this post and follow Hrithik Garg 🚀 for more 😊

[#Frontend](#) [#JavaScript](#) [#React](#) [#InterviewExperience](#) [#WebDevelopment](#) [#SDE](#)

Important Frontend Concepts Checklist-

1. Pagination

2. Infinite Scrollbar

3. Debouncing

4. Websocket

5. REST vs GraphQL APIs

6. Local Storage vs Cookies

7. Authentication vs Authorization

8. Redux

9. Lazy Loading

10. Code Splitting

11. Bundle Size Optimization

12. Tree Shaking

13. Memoization (useMemo, useCallback)

14. Caching (Client + Server)

15. CSR vs SSR vs SSG vs ISR

16. Core Web Vitals (LCP, INP, CLS)

17. Cross Browser Compatibility

18. Optimistic UI Updates

19. Suspense (React)

20. Image Optimization (WebP, AVIF)

21. Accessibility (a11y)

22. Webpack

23. Micro-frontend Architecture

24. Testing- RTL, Jest, Playwright

25. Polyfills, Babel

Get ebook with (detailed 232 ques = 50+ Reactjs Frequent Ques & Answers, 45+ Reactjs scenario based questions, 90+ frequently asked interview questions and answers, 50+ Output based ques & ans, 25+ Coding Questions & ans)

Ebook Link: <https://lnkd.in/gJMmH-PF>

Interview Guidance: <https://lnkd.in/gr9PCuxd>

Follow on Instagram : <https://lnkd.in/g-iSTsRd>

[#javascript](#) [#javascriptdeveloper](#) [#reactjs](#) [#reactnative](#) [#vuejsdeveloper](#) [#angular](#)
[#angulardeveloper](#)

After going through 50+ interview experiences, I realized one thing:

Most people don't fail because of React...

They fail because their JavaScript is weak.

So I'm starting a 4-part series to help you prepare systematically:

📌 Phase 1 — JavaScript (Fundamentals + Core)

📌 Phase 2 — React (Beginner → Advanced)

📌 Phase 3 — SEO + Optimization

📌 Phase 4 — DSA for Frontend

🔥 PHASE 1 — JavaScript (Start here. No shortcuts.)

If your JS is strong, 50% of your interview is already cracked.

► How JS works (V8, Call Stack, Memory Heap)

► Hoisting, Scope, Lexical Environment

► var vs let vs const + TDZ

► Shadowing vs illegal Shadowing

- ▶ Closures (VERY IMPORTANT)
- ▶ First Class functions and Callback functions
- ▶ Event Loop (Microtask vs Macrotask)
- ▶ setTimeout & async behavior
- ▶ HOFs — map, filter, reduce, etc
- ▶ Function Currying
- ▶ Shallow vs Deep Copy
- ▶ Debouncing & Throttling
- ▶ Event Bubbling & Capturing
- ▶ Type Coercion & NaN
- ▶ Rest / Spread, Destructuring
- ▶ Objects vs Map vs Set
- ▶ ES6+ features
- ▶ localStorage vs sessionStorage vs IndexedDB
- ▶ async & defer
- ▶ Method Chaining
- ▶ Must practice output based questions in js

⚠ PHASE 2 (Core JS — Most people fail here)

- ▶ Promises, async/await
- ▶ Custom Promise
- ▶ Promise APIs (all, race, any, allSettled)
- ▶ this keyword
- ▶ call, apply, bind
- ▶ Prototypes & Inheritance
- ▶ Polyfills (must practice)



Best Resources for JavaScript:

- ▶ JavaScript Deep Dive: youtu.be/ZvbzSrg0afE
- ▶ Namaste Dev: namastedev.com
- ▶ Big Frontend: bigfrontend.dev/problem

I'll be posting Phase 2 (React) next.

If you're preparing for frontend interviews, follow along — this will help.



Repost to help someone who's preparing.

[#JavaScript](#) [#FrontendDevelopment](#) [#ReactJS](#) [#WebDevelopment](#) [#InterviewPrep](#)
[#CodingInterview](#) [#SoftwareEngineering](#) [#CareerGrowth](#)

Cracking Frontend Interviews in 2026 — Phase 3 (SEO + Performance) ⚡

The wait is over... here's Phase 3.

While preparing for frontend interviews, I realized something:

Most of us focus a lot on React, features, UI...

but very little on performance and SEO.

And surprisingly, this is where many candidates struggle in interviews.

🔥 PHASE 3 — SEO (Search Engine Optimization)

Even the best UI is useless if users can't find it.

- ▶ Types of SEO — On-page, Off-page, Technical
- ▶ Meta Tags, OG Tags (for sharing previews)
- ▶ Robots.txt & Sitemap.xml
- ▶ Canonical URLs (avoid duplicate content issues)
- ▶ Semantic HTML (header, main, article, section)
- ▶ Alt attributes (important for accessibility + SEO)
- ▶ Structured Data (Schema markup)

⚡ Core Web Vitals (VERY IMPORTANT in 2026)

These directly impact ranking + user experience:

- ▶ LCP — Largest Contentful Paint (loading speed)
- ▶ INP — Interaction to Next Paint (replaces FID)
- ▶ CLS — Cumulative Layout Shift (visual stability)
- ▶ TTFB — Time to First Byte

🚀 React App Performance Optimization

This is where interviews get practical.

- ▶ Code Splitting (React.lazy + dynamic imports)
- ▶ Lazy Loading (components, images)
- ▶ Bundle Optimization (Tree Shaking)
- ▶ Avoid unnecessary re-renders
- ▶ Memoization (useMemo, useCallback, React.memo)
- ▶ Virtualization (render 10k+ rows efficiently)
- ▶ Image Optimization (WebP, srcset, lazy loading)
- ▶ Caching Strategies (Browser, API, CDN)

► Web Workers (for heavy computations)

🧠 Real Interview Insight:

Most candidates can build features...

Very few can explain why their app is slow and how to fix it.

That's what differentiates you...

Next: Phase 4 — DSA for Frontend 💡

🔄 Repost to help someone preparing for interviews.

[#FrontendDevelopment](#) [#WebPerformance](#) [#SEO](#) [#ReactJS](#) [#JavaScript](#) [#SoftwareEngineering](#)
[#InterviewPrep](#) [#CareerGrowth](#)